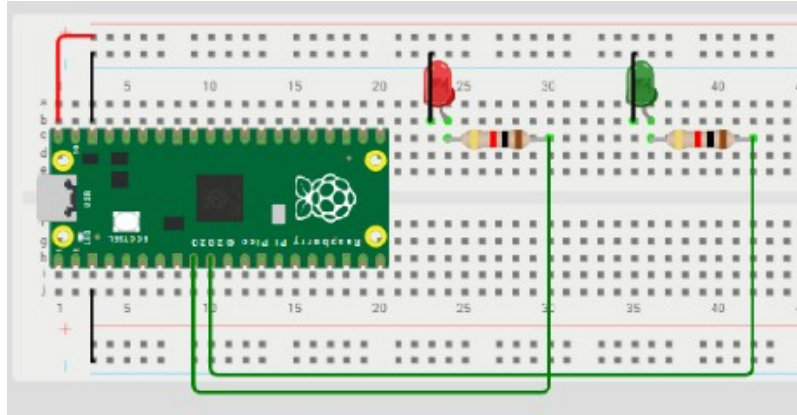


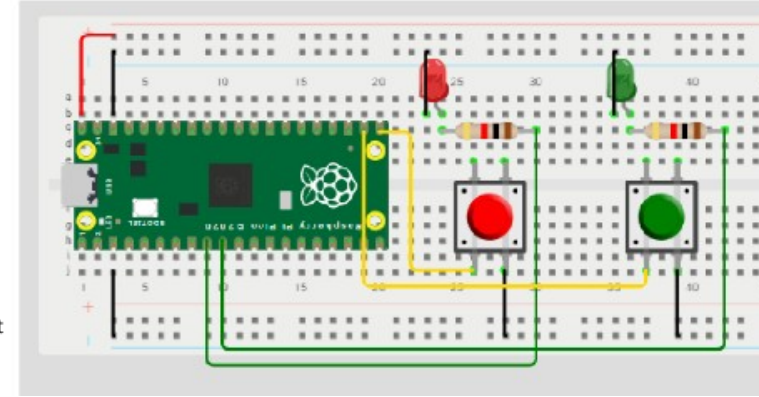
Codes Groupe 1 (Débutants)



Composant	GPIO
Led 1 (rouge)	6
Led 2 (verte)	7

Code couleur des câbles :

- Vcc : Rouge / Orange
- Gnd : Noir / Marron
- Données : Vert / Bleu / Jaune / Violet



Composant	GPIO
BP 1 (rouge)	16
BP 2 (vert)	17

test_led_simple.py

```
1  # on a besoin des fonctions "time" et "Pin" (qui est dans "machine") donc on les importe
2  import time
3  from machine import Pin
4
5  #On choisit le Pin sur lequel on va monter la LED et on lui dit que c'est une sortie
6  #(pas une entree de signal) c'est a dire que la sortie PIN va fournir l'energie pour commander la LED
7  LED = Pin(6, Pin.OUT)
8  #On donne la valeur 1 à la sortie pour allumer la LED
9  LED.value(1)
10 #On lui dit de rester allumée pendant 3 secondes
11 time.sleep(3)
12 #On lui dit de s'eteindre
13 LED.value(0)
14 #On lui dit de rester éteinte pendant 2 secondes
15 time.sleep(2)
```

test_bp_sans_irq.py

```
1  from machine import Pin
2  import time
3
4  bouton = Pin(16, machine.Pin.IN, Pin.PULL_UP)
5
6  while True:
7      if bouton.value() == 0:
8          print("Bouton Appuyé")
9      else:
10         print("Bouton Relaché")
11         time.sleep_ms(50)
```

test_led_fonction.py

```
1  import time
2  from machine import Pin
3
4  LED = Pin(6, Pin.OUT)
5
6  def test_led(led):
7      led.toggle()
8      time.sleep(0.5)
9
10 while True:
11     test_led(LED)
```

test_bp_avec_irq.py

```
1  from machine import Pin
2  import time
3
4  bouton = Pin(16, Pin.IN, Pin.PULL_UP)
5
6  date_dernier_appui = time.ticks_ms()
7  bouton_actif = False
8
9  # Fonction sera appelée lors de l'appui sur le bouton
10 def bouton_handler(pin):
11     global bouton_actif
12     global date_dernier_appui
13     # on evite les rebonds ...
14     if time.ticks_diff(time.ticks_ms(), date_dernier_appui) > 500:
15         bouton_actif = True
16         # on reinitialise la variable date_dernier_appui
17         date_dernier_appui = time.ticks_ms()
18
19 # Fonction qui associe l'appui sur le bouton a l'appel de la fonction ci dessus
20 bouton.irq(trigger = machine.Pin.IRQ_RISING, handler = bouton_handler)
21
22 def test_bouton():
23     global bouton_actif
24     # on attends 1ms
25     time.sleep(1)
26     # on regarde l'etat du bouton
27     if bouton_actif:
28         print("Bouton actif")
29         bouton_actif = False
30
31 print("En attente d'un appui sur le BP")
32 while True:
33     test_bouton()
```

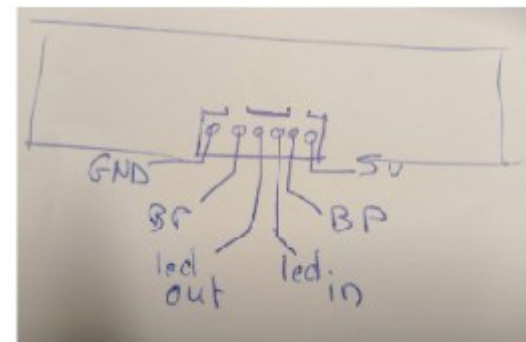
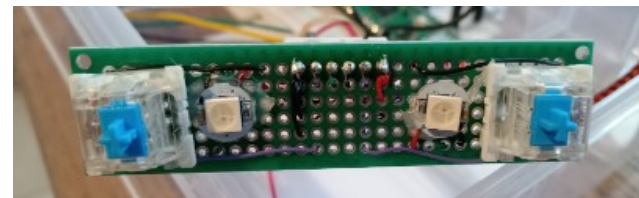
Codes Groupe 2 (Avancés)

test_neopixel_fill.py

```
1  from neopixel import Neopixel
2  import time
3
4  # declaration du ruban de leds
5  NUM_LED = 2
6  leds = Neopixel(NUM_LED, 0, 13, "GRB")
7
8  # on definit la luminosite des leds / 255
9  leds.brightness(15)
10 # on eteint toute les leds
11 leds.clear()
12 # on affiche
13 leds.show()
14
15
16 while True:
17     # on allume toutes les leds en rouge pendant 1 sec
18     leds.fill((255,0,0))
19     leds.show()
20     time.sleep(1)
21     # on allume toutes les leds en vert pendant 1 sec
22     leds.fill((0,255,0))
23     leds.show()
24     time.sleep(1)
25     # on allume toutes les leds en bleu pendant 1 sec
26     leds.fill((0,0,255))
27     leds.show()
28     time.sleep(1)
```

test_neopixel_set_pixel.py

```
1  from neopixel import Neopixel
2  import time
3
4  # declaration du ruban de leds
5  NUM_LED = 2
6  leds = Neopixel(NUM_LED, 0, 13, "GRB")
7
8  # declaration de quelques couleurs
9  couleur_rouge = (255,0,0)
10 couleur_vert = (0,255,0)
11 couleur_bleu = (0,0,255)
12 couleur_noir = (0,0,0)
13
14 # on definit la luminosite des leds
15 leds.brightness(15)
16 # on eteint toute les leds
17 leds.clear()
18 # on affiche
19 leds.show()
20
21 while True:
22     # on allume tour a tour les leds en bleu
23     for i in range (0,NUM_LED):
24         leds.set_pixel(i,couleur_bleu)
25         leds.show()
26         time.sleep(1)
27     time.sleep(1)
28     # on allume tour a tour les leds en vert
29     for i in range (0,NUM_LED):
30         leds.set_pixel(i,couleur_vert)
31         leds.show()
32         time.sleep(1)
33     time.sleep(1)
34     #leds.clear()
```



Composant	GPIO
BP 3	18
BP 4	19
Neopixel	13


```

1  from machine import Pin
2  import time
3  import random
4
5  # Declaration des boutons et des leds
6  leds = [
7      Pin(6, Pin.OUT), #rouge
8      Pin(7, Pin.OUT) #verte
9  ]
10
11  boutons = [
12      Pin(16, Pin.IN, Pin.PULL_UP), #bouton sous la led rouge
13      Pin(17, Pin.IN, Pin.PULL_UP) #bouton sous la led verte
14  ]
15
16  # Fonctions utiles pour simplifier la lisibilité du code
17  def allumer_led(index):
18      leds[index].on()
19      time.sleep(0.5)
20      leds[index].off()
21      time.sleep(0.2)
22
23  # On stocke également la gestion du bouton dans une fonction
24  # pour simplifier la lecture du code
25  def attendre_bouton():
26      while True:
27          for i, bouton in enumerate(boutons):
28              if bouton.value() == 0: # appui
29                  leds[i].on()
30                  while bouton.value() == 0:
31                      pass
32                  leds[i].off()
33                  time.sleep(0.1)
34                  return i
35
36  def afficher_sequence():
37      time.sleep(0.5)
38      for couleur in sequence_jeu:
39          allumer_led(couleur)
40
41  # Fonctions vides à compléter !!!!
42  def erreur():
43      # pass indique qu'on ne fait rien dans cette fonction pour l'instant
44      # et que c'est ok
45      pass
46
47  def victoire():
48      # pass indique qu'on ne fait rien dans cette fonction pour l'instant
49      # et que c'est ok
50      pass
51
52  # Initialisation
53  # On éteint toutes les leds
54  for i in range(2):
55      leds[i].off()
56  time.sleep(1)
57
58  # On les rallume pour vérifier qu'elles sont toutes ok
59  for i in range(2):
60      leds[i].on()
61  time.sleep(1)
62
63  # On les re éteint pour commencer à jouer
64  for i in range(2):
65      leds[i].off()
66  time.sleep(1)
67
68  # On vide la liste qui contient la sequence de jeu
69  # On va la construire pas à pas dans le code
70  sequence_jeu = []
71
72
73  # Programme principal
74  print("Jeu de Simon")
75
76  while True:
77      # Ajoute une nouvelle étape
78      sequence_jeu.append(random.randint(0, 1))
79
80      # Calcul du niveau selon la longueur de la sequence
81      niveau = len(sequence_jeu)
82      print("Niveau :", niveau)
83
84      # On joue la séquence
85      afficher_sequence()
86
87      # Vérification de la saisie du joueur
88      perdu = False
89      for couleur_attendue in sequence_jeu:
90          choix_joueur = attendre_bouton()
91          if choix_joueur != couleur_attendue:
92              print("Perdu ! Score :", len(sequence_jeu) - 1)
93              erreur()
94              sequence_jeu = [] #on recommence, on efface la sequence de jeu
95              break
96          else:
97              victoire()
98
99      # On attend un peu
100     time.sleep(1)

```

Jeu Simon 2Leds

```

1  # 🎮 DUEL DE RÉFLEXES
2  ## Principe du jeu
3  ## Une LED va s'allumer au hasard :
4  ## 🔴 LED rouge → appuyer sur le bouton rouge
5  ## 🟢 LED verte → appuyer sur le bouton vert
6  ## Le joueur doit être **le plus rapide possible** !
7  ## Le programme mesure son **temps de réaction**.
8
9  from machine import Pin
10 import time
11 import random
12
13 ## Declaration des boutons et des leds
14 leds = [
15     Pin(6, Pin.OUT),      # rouge
16     Pin(7, Pin.OUT)       # verte
17 ]
18
19 boutons = [
20     Pin(16, Pin.IN, Pin.PULL_UP), # bouton rouge
21     Pin(17, Pin.IN, Pin.PULL_UP)  # bouton vert
22 ]
23
24 ## Fonction: Attendre qu'un bouton soit appuyé
25 def attendre_bouton():
26     while True:
27         for i, bouton in enumerate(boutons):
28             if bouton.value() == 0:
29                 return i
30
31 ## Fonction: Éteindre les LEDs
32 def eteindre_leds():
33     for led in leds:
34         led.off()

```

```

36 ## Programme principal
37 print("DUEL DE REFLEXES")
38
39 # Test des LEDs
40 for led in leds:
41     led.on()
42     time.sleep(1)
43     eteindre_leds()
44     time.sleep(1)

```

Jeux Reflexe 2Leds

```

46 while True:
47
48     print("PREPAREZ-VOUS...")
49     # On attend un peu avant de lancer le jeu
50     time.sleep(random.uniform(1, 3))
51     # Choisir une LED au hasard
52     led_choisie = random.randint(0, 1)
53     # Allumer la LED
54     leds[led_choisie].on()
55
56     # On commence à mesurer le temps
57     debut = time.ticks_ms()
58
59     # Attendre le bouton du joueur
60     choix_joueur = attendre_bouton()
61
62     # Calculer le temps de réaction
63     fin = time.ticks_ms()
64     temps_reaction = time.ticks_diff(fin, debut)
65
66     # Éteindre les LEDs
67     eteindre_leds()
68
69     # Vérification
70     if choix_joueur == led_choisie:
71         print("BRAVO !")
72         print("Temps de réaction :", temps_reaction, "ms")
73     else:
74         print("MAUVAIS BOUTON !")
75     time.sleep(2)

```

```

1  from machine import Pin
2  from neopixel import Neopixel
3  import time
4  import random
5
6  # Declaration des boutons et des leds neopixel
7  NUM_LED = 2
8  leds = Neopixel(NUM_LED, 0, 13, "GRB")
9
10 boutons = [
11     Pin(18, Pin.IN, Pin.PULL_UP), #fil noir = LED#0
12     Pin(19, Pin.IN, Pin.PULL_UP) #fil vert = LED#1
13 ]
14
15 # declaration de quelques couleurs
16 couleur_rouge = (255,0,0)
17 couleur_vert = (0,255,0)
18 couleur_bleu = (0,0,255)
19 couleur_noir = (0,0,0)
20
21 # on definit la luminosite des leds
22 leds.brightness(15)
23 # on eteint toute les leds
24 leds.clear()
25 # on affiche
26 leds.show()
27
28 # Fonctions utiles pour simplifier la lisibilité du code
29 def allumer_led(index):
30     leds.set_pixel(index,couleur_bleu)
31     leds.show()
32     time.sleep(0.5)
33     leds.clear()
34     leds.show()
35     time.sleep(0.2)

```

```

37 # On stocke également la gestion du bouton dans une fonction
38 # pour simplifier la lecture du code
39 def attendre_bouton():
40     while True:
41         for i, bouton in enumerate(boutons):
42             if bouton.value() == 0: # appui
43                 leds.set_pixel(i,couleur_bleu)
44                 leds.show()
45                 while bouton.value() == 0:
46                     pass
47                 leds.clear()
48                 leds.show()
49                 time.sleep(0.1)
50                 return i
51
52 def afficher_sequence():
53     time.sleep(0.5)
54     for couleur in sequence_jeu:
55         allumer_led(couleur)
56
57 # Fonctions vides à compléter !!!!
58 def erreur():
59     # pass indique qu'on ne fait rien dans cette fonction pour l'instant
60     # et que c'est ok
61     pass
62
63 def victoire():
64     # pass indique qu'on ne fait rien dans cette fonction pour l'instant
65     # et que c'est ok
66     pass
67
68 # Initialisation
69 # On éteint toutes les leds
70 leds.clear()
71 leds.show()
72 time.sleep(1)
73
74 # On les rallume pour vérifier qu'elles sont toutes ok
75 leds.fill(couleur_bleu)
76 leds.show()
77 time.sleep(1)
78
79 # On les re éteint pour commencer à jouer
80 leds.clear()
81 leds.show()
82 time.sleep(1)

```

Jeu Simon Neopixel

```

84 # On vide la liste qui contient la sequence de jeu
85 # On va la construire pas à pas dans le code
86 sequence_jeu = []
87
88
89 # Programme principal
90 print("Jeu de Simon")
91
92 while True:
93     # Ajoute une nouvelle étape
94     sequence_jeu.append(random.randint(0, 1))
95
96     # Calcul du niveau selon la longueur de la sequence
97     niveau = len(sequence_jeu)
98     print("Niveau :", niveau)
99
100     # On joue la séquence
101     afficher_sequence()
102
103     # Vérification de la saisie du joueur
104     perdu = False
105     for couleur_attendue in sequence_jeu:
106         choix_joueur = attendre_bouton()
107         if choix_joueur != couleur_attendue:
108             print("Perdu ! Score :", len(sequence_jeu) - 1)
109             erreur()
110             sequence_jeu = [] #on recommence, on efface la sequence de jeu
111             break
112         else:
113             victoire()
114
115     # On attend un peu
116     time.sleep(1)

```

Jeu Reflexe Neopixel

```

1  # 🎮 DUEL DE RÉFLEXES
2  ## Principe du jeu
3  ## Une LED va s'allumer au hasard :
4  ## 🔴 LED rouge → appuyer sur le bouton rouge
5  ## 🟢 LED verte → appuyer sur le bouton vert
6  ## Le joueur doit être **le plus rapide possible** !
7  ## Le programme mesure son **temps de réaction**.
8
9  from machine import Pin
10 from neopixel import Neopixel
11 import time
12 import random
13
14 ## Declaration des boutons et des leds
15 NUM_LED = 2
16 leds = Neopixel(NUM_LED, 0, 13, "GRB")
17
18 boutons = [
19     Pin(18, Pin.IN, Pin.PULL_UP), #fil noir = LED#0
20     Pin(19, Pin.IN, Pin.PULL_UP) #fil vert = LED#1
21 ]
22
23 # declaration de quelques couleurs
24 couleur_rouge = (255,0,0)
25 couleur_vert = (0,255,0)
26 couleur_bleu = (0,0,255)
27 couleur_noir = (0,0,0)
28
29 # on definit la luminosite des leds
30 leds.brightness(15)
31 # on eteint toute les leds
32 leds.clear()
33 # on affiche
34 leds.show()
35
36 ## Fonction: Attendre qu'un bouton soit appuyé
37 ✓ def attendre_bouton():
38     while True:
39         for i, bouton in enumerate(boutons):
40             if bouton.value() == 0:
41                 return i
42
43 ## Fonction: Éteindre les LEDs
44 def eteindre_leds():
45     leds.clear()
46     leds.show()
47
48 ## Programme principal
49 print("DUEL DE REFLEXES")
50
51 # Test des LEDs
52 leds.fill(couleur_bleu)
53 leds.show()
54 time.sleep(1)
55 eteindre_leds()
56 time.sleep(1)
57
58 while True:
59
60     print("PREPAREZ-VOUS...")
61     # On attend un peu avant de lancer le jeu
62     time.sleep(random.uniform(1, 3))
63     # Choisir une LED au hasard
64     led_choisie = random.randint(0, 1)
65     # Allumer la LED
66     leds.set_pixel(led_choisie, couleur_bleu)
67     leds.show()
68
69     # On commence à mesurer le temps
70     debut = time.ticks_ms()
71
72     # Attendre le bouton du joueur
73     choix_joueur = attendre_bouton()
74
75     # Calculer le temps de réaction
76     fin = time.ticks_ms()
77     temps_reaction = time.ticks_diff(fin, debut)
78
79     # Éteindre les LEDs
80     eteindre_leds()
81
82     # Vérification
83     if choix_joueur == led_choisie:
84         print("BRAVO !")
85         print("Temps de réaction :", temps_reaction, "ms")
86     else:
87         print("MAUVAIS BOUTON !")
88     time.sleep(2)

```